Advanced Simulation Methods

Ab Initio & MLIPs

Contents

1 Introduction **1**
- 1.1 Managing Workflows . . . 2
- 1.2 Setup Instructions . . . 4

2 Tasks **4**
- 2.1 Sampling Training Data . . . 5
- 2.2 DFT . . . 5
- 2.3 Training . . . 8
- 2.4 MD & Post-Processing . . . 8
- 2.5 Bonus Tasks . . . 9

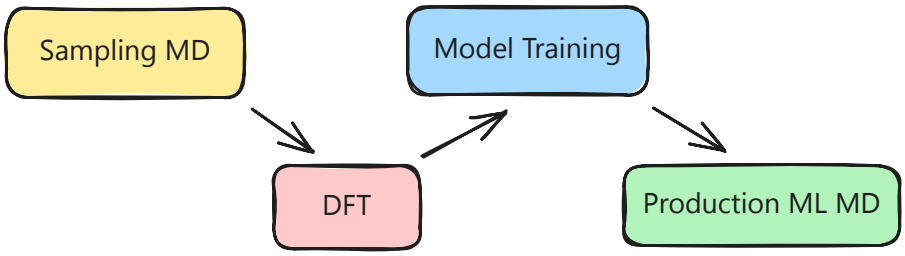
1 Introduction

This worksheet introduces you to building machine-learned interatomic potentials (MLIPs) at an *ab initio* level of accuracy. You will generate a training dataset with a pre-trained foundation model, relabel it with *ab initio* calculations, fit your own MLIP to that data, and finally use the MLIP to run large-scale molecular dynamics (MD) simulations.

The goal of this worksheet is to deepen your understanding of:

- generating diverse datasets for model training,
- selecting and preprocessing training data,
- setting up and tuning DFT calculations,
- basic hyperparameter tuning for ML models,
- post-processing of MD trajectories.

Liquid methanol serves as our model system throughout. The diagram below summarises the workflow.



1.1 Managing Workflows

Instead of orchestrating all of the steps above by hand, you will use a Python-based workflow manager:

1. **ZnTrack** – for reproducible, version-controlled workflow management.
2. **IPSuite** – a package built on top of ZnTrack that provides ready-made building blocks (*nodes*) for common atomistic-simulation tasks.

At its core, ZnTrack represents a computation as a directed acyclic graph (DAG). Each *node* is a self-contained step (for example a DFT single-point, a random selection, or a training run), and edges carry the data or metadata that one node passes to the next. Because every node declares its inputs, parameters and outputs, ZnTrack can detect which parts of the workflow are already up to date and only re-execute what has actually changed. ZnTrack builds on the data-version-control tool DVC and uses `git` to version both your code and your results.

IPSuite extends ZnTrack with a user-friendly interface to scientific software such as CP2K, apax and MACE, and provides utilities for working with molecular systems and machine-learned interatomic potentials.

Below is a minimal example of a ZnTrack + IPSuite workflow. It generates methanol conformers from a SMILES string and packs them into a simulation box with Packmol:

```python
import ipsuite as ips

project = ips.Project()  # one Project holds the whole workflow

with project:  # nodes defined inside this block are added to the graph
    methanol = ips.Smiles2Conformers(
        smiles="CO",
        numConfs=100,
    )
    box = ips.MultiPackmol(
        data=[methanol.frames],
        count=[32],
        density=792,          # kg/m^3
        n_configurations=1,
    )

```

```
project.build()  # write the DAG to disk (does not run it yet)
```

Running `python main.py` only *defines* the workflow; it does not execute the nodes. To actually run the computations, use the DVC command line:

```
$ dvc repro
```

A major convenience of this setup is that nodes are cached: a node is only re-executed if its inputs or parameters have changed. You can therefore iterate quickly on later steps without recomputing earlier ones.

TIP

Use `with project.group("group_name"):` to group related nodes together. Groups can be nested with multiple strings, e.g. `project.group("main", "subgroup")`. Groups are optional but keep large projects organised.

A few CLI commands that are useful while developing a workflow:

```
$ dvc dag       # visualise the dependency graph
$ zntrack list  # list all nodes in the project
$ dvc status    # check which nodes are out of date
```

TIP

To checkpoint your workflow, commit both the code and the DVC metadata with `git`:

```
$ git add .
$ git commit -m "Checkpoint: CP2K reference calculations done"
```

To roll back to an earlier state, find the commit you want and let DVC restore the associated data:

```
$ git log                # find the commit hash
$ git checkout <commit hash>
$ dvc checkout           # restore the cached outputs for that commit
```

Good moments to create a checkpoint are:

- after a long-running task (DFT, training) has succeeded,
- before a major edit to `main.py` or the workflow structure.

RESOURCES

- https://zntrack.readthedocs.io/en/latest/
- https://dvc.org/doc
- https://ipsuite.readthedocs.io/en/latest/
- https://mlipx.readthedocs.io/en/latest/

1.2 Setup Instructions

1. Install uv (a fast Python package manager) and clone the exercise repository:

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
$ git clone
↪  https://gitlab.icp.uni-stuttgart.de/fzills/advanced-simulation-methods-mlips adv_smp
```

2. Change into the cloned directory and install the Python dependencies:

```
$ cd adv_smp
$ uv sync
$ source .venv/bin/activate
```

3. Build the workflow graph by running the Python file:

```
$ python main.py
```

TIP

If you are not working in your home directory but for example on `/work` or `/data`, you may want to redirect the uv cache: set `UV_CACHE_DIR="/work/<username>/uv-cache"` or consult the uv documentation.

CP2K is not available as a `pip`-installable package. The necessary binaries are provided with the repository and are picked up automatically when the corresponding nodes are executed.

2 Tasks

TASK

Why would you use a workflow manager instead of a plain Python or Bash script? What specific benefits does this setup provide?

2.1 Sampling Training Data

Our final goal is an MLIP that predicts *ab initio* energies and forces at a fraction of the cost of DFT. In order to reach that accuracy, the model must be trained on a dataset of configurations that have been labelled with a reference *ab initio* method such as Hartree–Fock or DFT. Generating such a dataset directly from *ab initio* MD (AIMD) is usually infeasible because of the enormous computational cost.

Traditionally, one would have used a classical force field with a molecular dynamics engine to generate configurations, and this is still a common approach. More recently, however, so-called *foundation models* have become available: MLIPs that were pre-trained on very large and diverse datasets and can simulate a wide range of systems out of the box. In this worksheet we use the foundation model MACE to produce a trajectory of liquid methanol. From that trajectory we randomly select configurations, relabel them with DFT and add a Grimme dispersion correction. The resulting dataset is then used to train an MLIP, which predicts energies accurately and, via the Hellmann–Feynman theorem, forces as well. Once trained, this MLIP can drive MD simulations that are orders of magnitude faster than AIMD.

TASK

- How does MACE differ from the model you will train yourself?
- Why train your own model at all, given that foundation models already exist?

As a first step you need to create a cubic box of liquid methanol. The minimal workflow shown above is a good starting point.

TASK

- Adapt and execute the `main.py` script to generate initial configurations with Packmol.
- What are SMILES strings?
- Which temperatures would you simulate at, and why?
- Run a 1 ns MD simulation using the MACE foundation model.
- Load the resulting MACE trajectory with `zntrack.from_rev()` and inspect the `node.frames` list. Familiarise yourself with the ASE `Atoms` object.

RESOURCES

- https://ase-lib.org/ase/atoms.html

2.2 DFT

Next, you will compute accurate *ab initio* energies and forces with CP2K. A baseline configuration is provided in `config/cp2k.yaml`. The configurations on which to run DFT are passed to the node via its `data` argument. The basis-set and pseudopotential files used here come from the official CP2K repository and are shipped alongside this exercise.

The CP2K manual mostly refers to the legacy CP2K input format. Fortunately CP2K provides a toolset to convert between that legacy format and standard formats such as `*.json` or `*.yaml`, and we use YAML throughout this worksheet. To set up a DFT calculation we combine the `ips.CP2KModel` node (which builds the calculator) with `ips.ApplyCalculator` (which applies that calculator to a set of frames).

Before generating the full dataset we need to converge the integration grid. DFT accuracy depends sensitively on numerical settings such as the MGRID cutoff. For a training dataset this matters doubly: an MLIP can only be as consistent as the fluctuations in the data it is trained on. You will therefore run a small convergence test, varying the MGRID cutoff until energies and forces no longer change appreciably. For the analysis you will write a custom node `CompareDFT`. A scaffold is provided in `src/compare.py`; see the ZnTrack documentation for general information on writing custom nodes.

```python
from src import CompareDFT

with project:
    config = ips.RandomSelection(data=mace_md.frames, n_configurations=1)
    dft1 = ips.CP2KSinglePoint(data=config.frames, ...)
    dft2 = ...
    compare = CompareDFT(data=[dft1.frames, dft2.frames], mgrids=[..., ...])
```

TASK

- What does the MGRID parameter control in CP2K?
- Perform a convergence test: run multiple DFT calculations with varying MGRID cutoff on a small number of configurations from the MACE trajectory, until the change in forces and energies is below 0.002 (in the respective units).
- Modify `CompareDFT` in `src/compare.py` to analyse and plot the convergence.

Once you have chosen a suitable MGRID cutoff, you can relabel a subset of the MACE trajectory with DFT. Use `ips.RandomSelection` to draw a pool of configurations from the MACE MD that is large enough to act as the union of your future test and training sets – pick the size by estimating the DFT cost from the convergence test. Splitting this pool later (rather than running more DFT on demand) keeps the total DFT cost fixed while letting you vary the training-set size freely.

```python
with project:
    cp2k_model = ips.CP2KModel(
        config="config/cp2k.yaml",
        files=["config/BASIS_MOLOPT", "config/GTH_POTENTIALS"],
    )
    cp2k_data = ips.ApplyCalculator(
        data=selection.frames,
        model=cp2k_model,
```

```
        dump_rate=1,
    )
```

To add the Grimme D3 dispersion correction on top of the DFT energies and forces, use the `ips.TorchDFTD3` node from IPSuite:

```
with project:
    d3_calc = ips.TorchDFTD3(
        xc=...,
        damping=...,
        cutoff=...,
        cnthr=...,
        dtype=...,
        skin=...,
    )
    dataset = ips.ApplyCalculator(
        data=cp2k_data.frames,
        model=d3_calc,
        dump_rate=1,
        additive=True,
    )
```

TASK

- Use your optimised DFT settings to generate the training and test datasets.
- What are the core differences between LDA and GGA exchange–correlation functionals?
- Which of the two would you pick for methanol, and why?
- What is the role of dispersion corrections in DFT?
- Why can it be useful to evaluate the D3 correction outside of CP2K?

RESOURCES

- https://manual.cp2k.org/
- https://github.com/cp2k/cp2k-input-tools

TIP

Create a `git` commit once your DFT dataset has been computed successfully, so that you cannot accidentally lose this expensive result.

2.3 Training

The `apax` package provides a fast and accurate MLIP based on Gaussian Moment Neural Networks. It is configured via a `*.yaml` file; an example is provided in `config/apax.yaml`. `apax` exposes several nodes that integrate with IPSuite.

Pick a fixed test set, e.g. with `ips.RandomSelection` on the DFT pool. For the training set, select from the `excluded_frames` attribute of the first selection, i.e. pass `data=test_data.excluded_frames` into a second `ips.RandomSelection`. This keeps the total number of DFT calculations constant while letting you vary the training-set size.

TASK

- Try different hyperparameters and explain their impact:
  - the number of training configurations,
  - the radial cutoff `r_max`,
  - the number of basis functions,
  - the network architecture,
  - or any other setting from the documentation that you find worth exploring.
- Train ensemble models and compare the resulting uncertainties.
- Evaluate the MLIPs on the fixed test set.
- Select your best set of parameters and train two models: one on the raw DFT data, and one on the D3-corrected data.
- What are the benefits of model ensembles over single models?

RESOURCES

- `https://apax.readthedocs.io`

2.4 MD & Post-Processing

You can now use the trained MLIP to run large-scale simulations with close to *ab initio* accuracy. The `apax` package integrates with JaxMD for the dynamics, and IPSuite provides a structure-relaxation node to prepare the starting configuration.

TASK

- Build a larger box containing 64 methanol molecules with `ips.Smiles2Conformers` and `ips.MultiPackmol`. Relax the structure with `ips.ASEGeoOpt` and your trained model until the maximum force is below 0.05 eV/Å.
- Why is a structure relaxation important before running MD?
- Run a long-timescale MD simulation with `apax.nodes.ApaxJaxMD`.

- What timestep should you choose, and why is the choice important?
- Compare your simulation to the MACE reference simulation.
- Compute:
  - the self-diffusion coefficient (from the centre-of-mass motion),
  - radial distribution functions.

TIP

If something in the trajectory looks wrong, visualise it. MLIPs can easily produce unphysical behaviour when they are forced to extrapolate outside of their training data. If you cannot get a stable model, ask your tutor for additional training data.

RESOURCES

- https://apax.readthedocs.io/en/latest/configs/full_configs.html#molecular-dynamics-config

2.5 Bonus Tasks

TASK

- Compare different data-selection strategies.
- Compare different basis sets.
- If you want to contribute fixes or improvements to any of the `zincware` packages, talk to your tutor or open a pull request directly on GitHub. You can also work against a local clone of a package and include it with `uv add --editable /path/to/package`.